p0856r0
Restrict Access Property for `mdspan` and `span`

Published Proposal, 12 February 2018

This version:
https://kokkos.github.io/array_ref/proposals/P0856.html

Authors:
David S. Hollman
H. Carter Edwards
Christian Trott

Audience:
LEWG

Project:
ISO JTC1/SC22/WG21: Programming Language C++

Abstract

This paper proposes a semantic for the use of the property customization point on `span` and `mdspan` to express non-aliasing semantics analogous to those of the ISO-C keyword `restrict`. Reconceptualizing non-aliasing semantics as a customization point on a library feature addresses many of the previous ambiguities in non-aliasing semantics for C++, and the implications of this new approach are discussed.

Table of Contents

§ 1. Revision History

§ 1.1. P0856r0

Address 2017-11-Albuquerque LEWG feedback to generate paper for *restrict_access* property for `mdspan` and `span`.

§ 2. Motivation

The restrict (non-aliasing) property is a widely useful mechanism to enable array access optimization. This functionality is part of the C standard, several C++ compilers' extensions, and native to FORTRAN array semantics.

While the utility of non-aliasing type annotations is well-established in the literature, it is useful to reproduce several common motivating examples here to illustrate the utility in the context of `span` and `mdspan`.

Linear algebra produces one of the most compelling needs for non-aliasing semantics. Even the simplest operations, such as vector addition, suffer noticable degredation in the absense of a non-aliasing guarantee:

```
// Performce element-wise dest = a + b
template <typename T>
void element_product(span<const T> a, span<const T> b, span<T> dest) {
  assert(a.size() == b.size() && b.size() == c.size());
  for(typename span<T>::size_type i = 0; i < a.size(); ++i) {
    dest[i] = a[i] + b[i];
  }
}
```

Without `restrict` annotations, the compiler cannot vectorize this code, since `a` or `b` could point to the same memory as `dest`. With `dest` being changed every iteration, the compiler needs to preform fresh loads `a[i]` and `b[i]` every iteration in case, for instance, they point to `dest[i-1]`.

Reordering for the purposes of optimizing memory operations is another example of an optimization that `restrict` enables. Consider the (albeit naïve) implementation of a function that initializes two arrays at the start of some user's program:

```
void initialize_data(span<int> v0, span<int> v1) {
  for(typename span<int>::size_type i = 0; i < v0.size(); ++i) {
      v0[i] = 0;
      v1[i] = 1;
  }
}
```

In the absence of `restrict` information, the compiler cannot tell at what point, if any, the statement `v1[i] = 1` starts to overwrite the effects of previous statements on `v0` (or vice versa). With non-aliasing information provided, however, the compiler can transform this function into two calls to `memset`, which would be much more efficient on most architectures. (It's clear to most people in this example that the user should have made this transformation to begin with, but with intervening work in between the two assignments, or other increased complexity, this may not be as clear).

We propose to add the `restrict_access` property to the set of admissible properties of `mdspan` and `span`.

This paper does not propose a strategy for subsequent extensibility. We identify open questions that must be addressed in the design of such a strategy. P0900 addresses the design of the customization

point utilized here in a more generic context.

§ 3. Background and Precedent

[N3538], [N3635], and [N3988] previously proposed restrict-like semantics as a language feature rather than a library feature. A discussion of how presenting `restrict` semantics as a library feature addresses many of the concerns previously raised about these papers is given below in §5.1 Why a library feature?

§ 3.1. ISO C Language Standard

The C language standard provides the `restrict` keyword in 6.7.3 of the WG14 document [WG14-N1570]. An informal definition is given in paragraph 8 of 6.7.3:

> An object that is accessed through a restrict-qualified pointer has a special association with that pointer. This association, defined in 6.7.3.1 below, requires that all accesses to that object use, directly or indirectly, the value of that particular pointer. The intended use of the `restrict` qualifier (like the `register` storage class) is to promote optimization, and deleting all instances of the qualifier from all preprocessing translation units composing a conforming program does not change its meaning (i.e., observable behavior).

A formal definition of `restrict` follows in 6.7.3.1 of [WG14-N1570]:

> Let `D` be a declaration of an ordinary identifier that provides a means of designating an object `P` as a restrict-qualified pointer to type `T`.
>
> If `D` appears inside a block and does not have storage class extern, let `B` denote the block. If `D` appears in the list of parameter declarations of a function definition, let `B` denote the associated block. Otherwise, let `B` denote the block of main (or the block of whatever function is called at program startup in a freestanding environment).
>
> In what follows, a pointer expression `E` is said to be *based on* object `P` if (at some sequence point in the execution of `B` prior to the evaluation of `E`) modifying `P` to point to a copy of the array object into which it formerly pointed would change the value of `E`. Note that "based" is defined only for expressions with pointer types.
>
> During each execution of `B`, let `L` be any lvalue that has `&L` based on `P`. If `L` is used to access the value of the object `X` that it designates, and `X` is also modified (by any means), then the following requirements apply: `T` shall not be const-qualified. Every other lvalue used to access the value of `X` shall also have its address based on `P`. Every access that modifies `X` shall be considered also to modify `P`, for the purposes of this subclause. If `P` is assigned the value of a pointer expression `E` that is based on another restricted pointer object `P2`, associated with block `B2`, then either the execution of `B2` shall begin before the execution of `B`, or the execution of `B2` shall end prior to the assignment. If these requirements are not met, then the behavior is undefined.
>
> Here an execution of `B` means that portion of the execution of the program that would correspond to the lifetime of an object with scalar type and automatic storage duration associated with `B`.
>
> A translator is free to ignore any or all aliasing implications of uses of `restrict`.

§ 3.1.1. Dependence of N4700 on ISO C `restrict`

Note that [N4700] already references the ISO C `restrict` keyword in paragraph 2 of [library.c], implying that C++ implementations should already be aware of its semantics (though, as stated in [WG14-N1570], conforming implementations can completely ignore `restrict`):

> The descriptions of many library functions rely on the C standard library for the semantics of those functions. In some cases, the signatures specified in this document may be different from the signatures in the C standard library, and additional overloads may be declared in this document, but the behavior and the preconditions (including any preconditions implied by the use of an ISO C `restrict` qualifier) are the same unless otherwise stated.

§ 3.2. GCC's `__restrict__`

GCC implements restricted access in C++ via the `__restrict__` (or `__restrict`) compiler-specific extension. The compiler documentation [GCCRestrictDoc] provides the following description:

> In addition to allowing restricted pointers, you can specify restricted references, which indicate that the reference is not aliased in the local context.
>
> ```
> void fn (int *__restrict__ rptr, int &__restrict__ rref)
> {
>   /* … */
> }
> ```
>
> In the body of `fn`, `rptr` points to an unaliased integer and `rref` refers to a (different) unaliased integer.
>
> You may also specify whether a member function's this pointer is unaliased by using `__restrict__` as a member function qualifier.
>
> ```
> void T::fn () __restrict__
> {
>   /* … */
> }
> ```
>
> Within the body of `T::fn`, this has the effective definition `T *__restrict__ const this`. Notice that the interpretation of a `__restrict__` member function qualifier is different to that of `const` or `volatile` qualifier, in that it is applied to the pointer rather than the object. This is consistent with other compilers that implement restricted pointers.
>
> As with all outermost parameter qualifiers, `__restrict__` is ignored in function definition matching. This means you only need to specify `__restrict__` in a function definition, rather than in a function prototype as well.

It is clear that GCC's implementation of this extension would be sufficient to implement the proposed feature trivially. Tests with gcc 7.3 and subsequent analysis of the machine code produced during compilation indicates that the existing extension would support the semantics proposed herein and perform the expected optimizations consistent with analogous non-aliasing information on raw pointers.

§ 3.3. MSVC's `__declspec(restrict)`

MSVC provides a couple of similar compiler-specific extensions via the syntaxes `__declspec(restrict)` and `__restrict`. The compiler documentation [MSVCRestrictDoc] gives the following description of `__declspec(restrict)`:

> Applied to a function declaration or definition that returns a pointer type and tells the compiler that the function returns an object that will not be aliased with any other pointers.

The compiler documentation's description for `__restrict` indicates that it is similar to `__declspec(restrict)`, and, indeed, to ISO C `restrict`:

> `__restrict` is similar to restrict from the C99 spec, but `__restrict` can be used in C++ or C programs.

§ 3.4. Clang++ and the `noalias` Metadata in LLVM IR

LLVM's internal representation (IR) includes a rich set of memory aliasing metadata annotations [LLVMIRDoc], including the `noalias` metadata that expresses `restrict`-like semantics for function parameters (among other uses). Clang++ supports the `__restrict` and `__restrict__` extensions from GCC both in the Clang++ frontend and through these metadata attributes. The presence of these attributes in the LLVM IR means that other frontends that translate into LLVM IR will also have the ability to express these semantics.

§ 3.5. IBM XL C++

The documentation for IBM XL C++ [XLCRestrictDoc] indicates that it supports ISO C `restrict` semantics in C++ via the `__restrict` or `__restrict__` extensions.

§ 3.6. NVIDIA's nvcc

The nvcc compiler supports the `__restrict` or `__restrict__` extensions for the GPU and (as long as the underlying CPU compiler supports it) the CPU. [NVCCRestrict]

§ 4. Proposal

We propose utilitzing the customization point explored in P0900 to express the `restrict_access` property:

```
namespace std {
namespace experimental {
inline namespace fundamentals_v3 {

  // Trait common to all span and mdspan property proposals, as given in e.
  template< class T, class P >
  struct has_property;

  // Variable template for has_property
  template< class T, class P >
  inline constexpr bool has_property_v = has_property<T, P>::value ;

  // Tag class for indicating restrict access
  struct restrict_access ;

  // Specialization of has_property for restrict_access
  template< class T >
  struct has_property<T, restrict_access> {
    inline constexpr bool value = /* see below */;
  };

}}}
```

§ 4.1. `has_property<T, restrict_access>::value`

Evaluates to true if `T` is instantiation of `span` or `mdspan` and the `Properties...` parameter pack of `T` contains `restrict_access`.

§ 4.2. Application of the Restriction to `span`

Working relative to [P0122r5], a preliminary attempt at wording the restriction proposed herein is as follows (in addition to the changes proposed by [P0546r1]):

23.7.2.7 [span.restrict]

Let `S` be an instantiation of `span` such that `has_property<S,restrict_access>::value` is `true`, and let `s` be an instance of `S`. Let `p` be an object of type `S::pointer` and `sz` be an object of type `S::index_type`. The *restricted lifetime of* `s` *with respect to {* `p, sz` *}* is defined such that it either:

- begins with the construction of `s` and ends with the destruction of `s`
- begins with the construction of `s` and ends with `s` becoming an *xvalue*
- begins with the construction of `s` and ends with the first invocation of an assignment operator of `s`.
- begins with an invocation of an assignment operator of `s` and ends with the immediately subsequent invocation of an assignment operator of `s`.
- begins with an invocation of an assignment operator of `s` and ends with the destruction of `s`
- begins with an invocation of an assignment operator of `s` and ends `s` becoming an *xvalue*.

and has the property that `p == s.data()` and `sz == s.size()`. We abbreviate the "*restricted lifetime of* `s` *with respect to {* `p, sz` *}* with the notation `L{s, p, sz}`. The lifetime of `s` thus consists of a disjoint set of *restricted lifetimes* `L_1{s, p_1, sz_1}`, ..., `L_n{s, p_n, sz_n}`. The truth of `has_property<S,restrict_access>::value` implies that during any given restricted lifetime `L_i{s, p_i, sz_i}` of an instance `s` of `S`, no value of a pointer or address of a reference may be used to form a glvalue expression that modifies an object with an address in the range `[p_i, p_i + sz_i)` except for those derived from:

- `s.begin()`
- `s.cbegin()`
- `s.rbegin()`
- `s.crbegin()`
- `s.operator[]()`
- `s.operator()()`
- `s.data()`
- `as_bytes(s)`
- `as_writable_bytes(s)`
- Any of these operations on a copy of `s` made during this restricted lifetime
- Any of these operations on a reference to `s` or the dereference of a pointer to `s`.

> - Any of these operations on a subspan derived from `s`
>
> These restrictions apply to any modifying accesses indeterminately sequenced with the beginning or end of the given restricted lifetime `L_i`. Any other accesses that modify the referenced memory through any other means result in undefined behavior. Additionally, accessing any object not in the range `[p_i, p_i + sz_i)` via a pointer or reference derived from any of the above sources results in undefined behavior.

Formal wording for the application of the restriction to `mdspan` will follow discussion of the content in the current paper, but it likely to be similar to the wording for `span` above.

§ 5. Discussion

§ 5.1. Why a library feature?

From [N3988] itself and from the on-the-record discussion of it at the Rapperswil meeting in 2014, here are some of the objections raised about restrict-like semantics in C++:

- Unclear whether it should affect name mangling
  - As a library feature, the name mangling behavior is well-defined and obvious
- Unclear whether it should affect overloading
  - As a library feature, the overload behavior is well-defined and obvious. It can also be manipulated in the same way as any other template parameter, for instance using SFINAE.
- `restrict`, as given in ISO C, has insufficient expressiveness (you cannot precisely enough describe what is restricted)
  - The extensibility of `Properties...` for `span` and `mdspan` as a library feature provides a clear path forward towards addressing this.
- In C, the lifetime of the `restrict` contract is awkward to express and therefore is restricted to initialization.
  - Object lifetime of a library wrapper provides clear bounds for the contract.
- Semantics for implicit or explicit capture of `restrict` pointers in lambdas is unclear.
  - As a library feature, the object will not change type during capture, therefore the transfer of the restrict contract is immediately clear.

Active content removed

§ References

§ Informative References

[GCCRestrictDoc]
Restricted Pointer Aliasing. URL: https://gcc.gnu.org/onlinedocs/gcc/Restricted-Pointers.html

[LLVMIRDoc]
LLVM Language Reference Manual. URL: https://llvm.org/docs/LangRef.html

[MSVCRestrictDoc]
restrict. URL: https://docs.microsoft.com/en-us/cpp/cpp/restrict

[N3538]
Lawrence Crowl. Pass by Const Reference or Value. 6 March 2013. URL: https://wg21.link/n3538

[N3635]
M. Wong, R. Silvera, R. Mak, C. Cambly, et al.. Towards restrict-like semantics for C++. 29 April 2013. URL: https://wg21.link/n3635

[N3988]
H. Finkel, H. Tong, et al.. Towards restrict-like aliasing semantics for C++. 23 May 2014. URL: https://wg21.link/n3988

[N4700]
Richard Smith. Working Draft, Standard for Programming Language C++ Note:. URL: https://wg21.link/n4700

[NVCCRestrict]
Jeremy Appleyard. CUDA Pro Tip: Optimize for Pointer Aliasing. URL: https://devblogs.nvidia.com/parallelforall/cuda-pro-tip-optimize-pointer-aliasing/

[P0122r5]
Neil MacIntosh. span: bounds-safe views for sequences of objects. URL: https://wg21.link/p0122r5

[P0546r1]
Carter Edwards, Bryce Lelbach. Span - foundation for the future. URL: https://wg21.link/p0546r1

[WG14-N1570]
Committee Draft, Programming Languages — C. URL: http://www.open-std.org/jtc1/sc22/wg14/www/docs/n1570.pdf

[XLCRestrictDoc]

The restrict type qualifier. URL: https://www.ibm.com/support/knowledgecenter/en/SS2LWA_12.1.0/com.ibm.xlcpp121.bg.doc/language_ref/restrict_type_qualifier.html